18 · HttpClient and Working with APIs

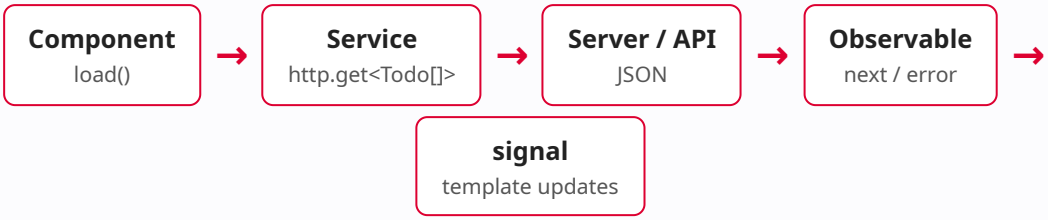
Intermediate Angular learning notes

a. Definition

HttpClient is Angular's service for talking to servers over HTTP (REST APIs). Enable it with `provideHttpClient()`, inject it, and call `get`, `post`, `put`, `delete`. Each call returns an **Observable**: nothing is sent until you `subscribe`.

Real apps always handle three states: **loading** (waiting), **success** (data arrived) and **error** (network or server failure). Put API calls in a **service**, and give request/response data a TypeScript interface.

b. Diagram



Request goes out; the response (or error) comes back later

c. Syntax

```
provideHttpClient()                       // app.config.ts

private http = inject(HttpClient);
this.http.get<Todo[]>(url).subscribe({
  next:  (data) => ...,
  error: (err)  => ...,
});
this.http.post<Todo>(url, body)
```

d. Example Program

Programs/18_HttpClient_and_APIs/todo-api.component.ts

```
import { HttpClient, provideHttpClient } from '@angular/common/http';
import { ApplicationConfig, Component, inject, Injectable, signal } from '@angular/core';

export const appConfig: ApplicationConfig = {
  providers: [provideHttpClient()], // enables HttpClient
};

interface Todo {
  id: number;
  title: string;
  done: boolean;
}

// All API calls live in a service
@Injectable({ providedIn: 'root' })
export class TodoApi {
```

```
  private http = inject(HttpClient);
  private base = 'https://api.example.com'; // replace with your real API

  list() {
    return this.http.get<Todo[]>(`${this.base}/todos`);
  }
  add(title: string) {
    return this.http.post<Todo>(`${this.base}/todos/add`, { title });
  }
  broken() {
    return this.http.get<Todo[]>(`${this.base}/broken`);
  }
}

@Component({
  selector: 'app-root',
  template: `
    <button id="load" (click)="load()">Load todos</button>
    <button id="add" (click)="add()">Add todo</button>
    <button id="broken" (click)="callBroken()">Call broken URL</button>

    @if (loading()) { <p>Loading...</p> }
    @if (error()) { <p>Error: {{ error() }}</p> }

    <ul>
      @for (t of todos(); track t.id) {
        <li>{{ t.done ? '[x]' : '[ ]' }} {{ t.title }}</li>
      }
    </ul>
  `,
})
export class TodoApiComponent {
  private api = inject(TodoApi);
  todos = signal<Todo[]>([]);
  loading = signal(false);
  error = signal('');

  load() {
    this.loading.set(true);
    this.error.set('');
    this.api.list().subscribe({
      next: (data) => {
        this.todos.set(data);
        this.loading.set(false);
      },
      error: () => {
        this.error.set('Could not load todos');
        this.loading.set(false);
      },
    });
  }

  add() {
    this.api.add('Buy jasmine flowers').subscribe((todo) => this.todos.update((list) =>
[...list, todo]));
  }

  callBroken() {
    this.error.set('');
    this.api.broken().subscribe({ error: () => this.error.set('Server returned an error
(500)') });
  }
}
```

e. Output

Rendered in the browser (text on screen, after the UI steps below):

1. Click **Load todos**, then **Add todo**.
2. Click **Call broken URL** to see error handling.

```
--- After load + add ---
Load todosAdd todoCall broken URL
[x] Learn Angular
[ ] Practice HttpClient
[ ] Buy jasmine flowers

--- After a failing request ---
Load todosAdd todoCall broken URL
Error: Server returned an error (500)
[x] Learn Angular
[ ] Practice HttpClient
[ ] Buy jasmine flowers
```

f. Real-World Example (English + தமிழ்)

English

In a restaurant you order masala dosa (request); the kitchen prepares it and sends it (response). While waiting you see 'preparing' (loading); if batter is over you are told (error).

தமிழ்

நீங்கள் ஒரு உணவகத்தில் அமர்ந்து சர்வரிடம் 'ஒரு மசாலா தோசை' என்று ஆர்டர் கொடுக்கிறீர்கள் - இது **request**. சமையலறை (server) தோசை தயாரித்து அனுப்புகிறது - இது **response**. இது உடனே நடக்காது; நேரம் ஆகும் (asynchronous).

காத்திருக்கும்போது 'தயாராகிக்கொண்டிருக்கிறது...' (loading). தோசை தயாராகிவிட்டால் தட்டில் வரும் (success). 'மாவு தீர்ந்துவிட்டது' என்றால் (error) நமக்குத் தெரிவிக்க வேண்டும். `HttpClient` இந்த சர்வர் வேலையைச் செய்கிறது.

g. Pros

- Typed requests/responses
- Interceptors
- Easy testing with mocks
- Works with RxJS

h. Cons

- Observables must be subscribed
- Boilerplate for loading/error states

i. Advanced / New Features

- `provideHttpClient(withFetch())`

- Functional interceptors
- `httpResource` (experimental) for signal-based requests

j. Common Mistakes

- Not subscribing (or not using `async` pipe / `toSignal`); an HTTP Observable does nothing until subscribed.
- Ignoring errors; always handle the `error` callback and show a message.
- Forgetting `provideHttpClient()` (error: No provider for HttpClient).
- Calling the API directly from many components instead of in a shared service.

k. Interview Questions

Q1. What is HttpClient?
Angular's injectable service for HTTP requests, returning typed Observables.

Q2. Why must you subscribe to an HTTP call?
Observables are lazy; the request is sent only on subscription.

Q3. How do you handle errors?
Use the `error` callback in `subscribe`, or the RxJS `catchError` operator in the pipe.

Q4. How do you send headers or query params?
Pass an options object: `http.get(url, { headers, params })`.

l. Practice Exercises

1. Add a `delete(id)` method and a delete button for each todo.
2. Show a spinner text while loading using `loading()`.
3. Use `catchError` + `of([])` inside the service so the component never crashes on error.